FEATURE ENGINEERING USING PYTHON

# Data Types & Features

*Numerical · Categorical · Ordinal · Discrete · Continuous · Interval · Ratio*

# What is a Feature?

A feature is simply **a characteristic or property describing an object.**

- In Machine Learning, we select only the features that help prediction.
- Feature selection = choosing important features, removing unnecessary ones.
- Good features improve machine learning performance.
- Features are not all the same type — numbers, labels, rankings, grades, measurements.

**THE TRAP**

**Age = 25** and **Zip_Code = 201001**

Both look like numbers. Both are stored as int64 in pandas.

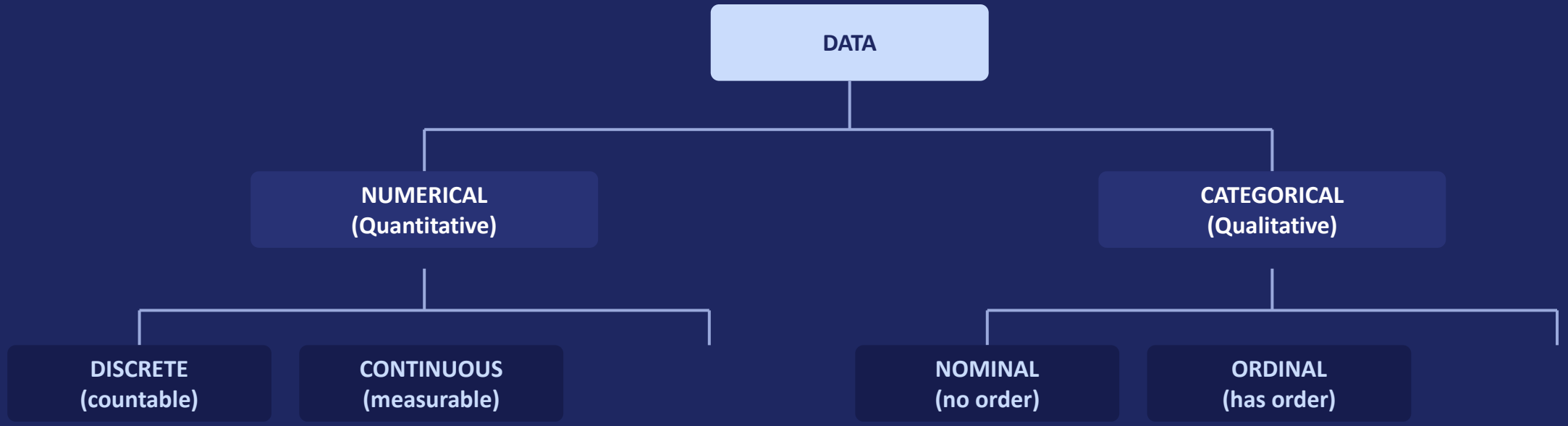
**Zip_Code 402002** is NOT “twice as much location” as **201001** — meaningless.

**Age 50** genuinely IS twice **Age 25** — valid.

***“Python doesn’t know the meaning of your data — only you do.”***

Wrong classification breaks scaling, encoding, aggregation, and model choice downstream.

# The Big Picture: Two Ways to Classify Data




## Levels of Measurement (Stevens' Scale)

| NOMINAL | → | ORDINAL | → | INTERVAL | → | RATIO |
|---|---|---|---|---|---|---|
| *name* | | *rank* | | *equal gaps, no true 0* | | *equal gaps, true 0 exists* |

# Numerical Data: Discrete vs Continuous

*Measurable quantities — arithmetic is meaningful on both.*

## Discrete

*"Can you have 2.5 children? No — you count children in whole numbers."*

EXAMPLES

- Num_Children
- Number_of_Products_Sold
- Number of Students

## Continuous

*"Can your height be 172.53 cm? Yes — height flows continuously."*

EXAMPLES

- Height_cm
- Weight_kg
- Salary

# Categorical Data: Nominal vs Ordinal

*Groups or labels — arithmetic is meaningless.*

## Nominal

*"Is 'HR' greater than 'IT'? No — they're just different buckets."*

EXAMPLES

- Gender
- Department
- Blood_Group

## Ordinal

*"We know PhD > Bachelor. But is High School→Bachelor the same gap as Bachelor→Master? We can't say."*

EXAMPLES

- Education_Level: High School < Bachelor < Master < PhD
- Performance_Rating: Poor < Average < Good < Excellent

# Interval Data — Equal Gaps, No True Zero

*Zero does not mean "absence of the thing" — it's just an arbitrary reference point.*

### Temperature (°C)

0°C = freezing point, not "no temperature." 40°C is NOT twice as hot as 20°C.

### IQ Score

IQ 150 is NOT '1.5× as intelligent' as IQ 100. Zero IQ isn't 'zero intelligence.'

### Calendar Years

Year 2000 → 2010 = 10 yrs, meaningful. But Year 0 ≠ "time did not exist."

### Floor Number

Floor 5 − Floor 1 = 4 floors. But Floor 0 ≠ "building doesn't exist."

**Rule of thumb:** Interval data supports addition and subtraction (differences) but NOT multiplication/division (ratios).

# Ratio Data — True Zero Exists

*Equal gaps AND a meaningful zero, so ratios ("twice as much") are valid.*

| FEATURE | WHAT DOES 0 MEAN? |
| --- | --- |
| Money in wallet | *No money* |
| Mobile data balance | *No internet data* |
| Number of students | *No students* |
| Distance travelled | *No distance* |
| Salary | *No salary* |
| Number of books | *No books* |
| Time spent studying | *No study time* |

## WORKED EXAMPLES

**Example 1**

Student A has ₹100, Student B has ₹200 → B has twice as much money as A. ₹0 = no money.

**Example 2**

Class A: 20 students, Class B: 40 students → Class B has twice the students. 0 students = no students.

**Example 3**

Daily data usage: 1 GB, 2 GB, 0 GB → 0 GB = no data used; 2 GB is genuinely twice 1 GB.

# Interval vs Ratio — and Why It All Matters

### Interval

- No true zero
- Example: Celsius, IQ

### Ratio

- True zero exists
- Example: Salary, Age

| Type | Order? | Equal Gaps? | True Zero? | Arithmetic ? |
|---|---|---|---|---|
| Nominal | No | No | No | No |
| Ordinal | Yes | No | No | Ranking only |
| Interval | Yes | Yes | No | + / − only |
| Ratio | Yes | Yes | Yes | + − × ÷ |

## Why This Matters for Feature Engineering

- **Encoding:** One-Hot for Nominal · Ordinal Encoding (with correct order) for Ordinal
- **Scaling:** StandardScaler / MinMaxScaler only valid for Interval / Ratio numerical data
- **Binning:** Discretization turns Continuous → Ordinal / Discrete
- **Aggregation:** Mean / Sum only valid for Ratio, sometimes Interval — never for Nominal

*Get the data type wrong at the start — and every downstream technique breaks.*

# Hands-On Lab: Detect the Type Trap

*Dataset: employee_data_types.csv*

```
import pandas as pd

df = pd.read_csv('employee_data_types.csv')
print(df.dtypes)
```

**OUTPUT (partial)**

```
Emp_ID          int64
Age             int64
Zip_Code        int64
Education_Level object
```

### What pandas got wrong

**Emp_ID** and **Zip_Code** are read as **int64** — but they are **Nominal**, not Ratio.

***"Pandas only sees storage format — not meaning. Classification is your job, not the machine's."***

Next: correct these columns to their true types.

# Hands-On Lab: Fix Nominal & Ordinal Columns

**Step 1 — Nominal columns → category**

```
nominal_cols = ['Gender', 'Department',
                'Emp_ID', 'Zip_Code']
for col in nominal_cols:
    df[col] = df[col].astype('category')
```

**Step 2 — Ordinal columns → ordered category**

```
education_order = ['High School', 'Bachelor',
                   'Master', 'PhD']

df['Education_Level'] = pd.Categorical(
    df['Education_Level'],
    categories=education_order,
    ordered=True
)
```

**Why ordered=True matters**

- Enables correct sorting: df.sort_values('Education_Level')
- Enables valid comparisons: df['Education_Level'] > 'Bachelor'
- Neither works on a plain string column, and One-Hot Encoding would destroy the order entirely.

**Verify:**

```
df['Education_Level'].cat.ordered
# True
```

# Hands-On Lab: Verify — Aggregation Rules

*Prove the theory: mean() is valid for Ratio data, meaningless for Nominal data.*

## Age (Ratio)

```
print(df['Age'].mean())

# 35.67
```

**MEANINGFUL**

True zero exists, equal gaps — the average age is a genuinely meaningful summary of the group.

## Zip_Code (Nominal)

```
print(df['Zip_Code'].mean())

# 201002.73  ← runs, but useless
print(df['Zip_Code'].value_counts())
# ← the CORRECT operation
```

**MEANINGLESS**

The number computes, but no zip code is ‘201002.73’ — use value_counts() for Nominal data instead.